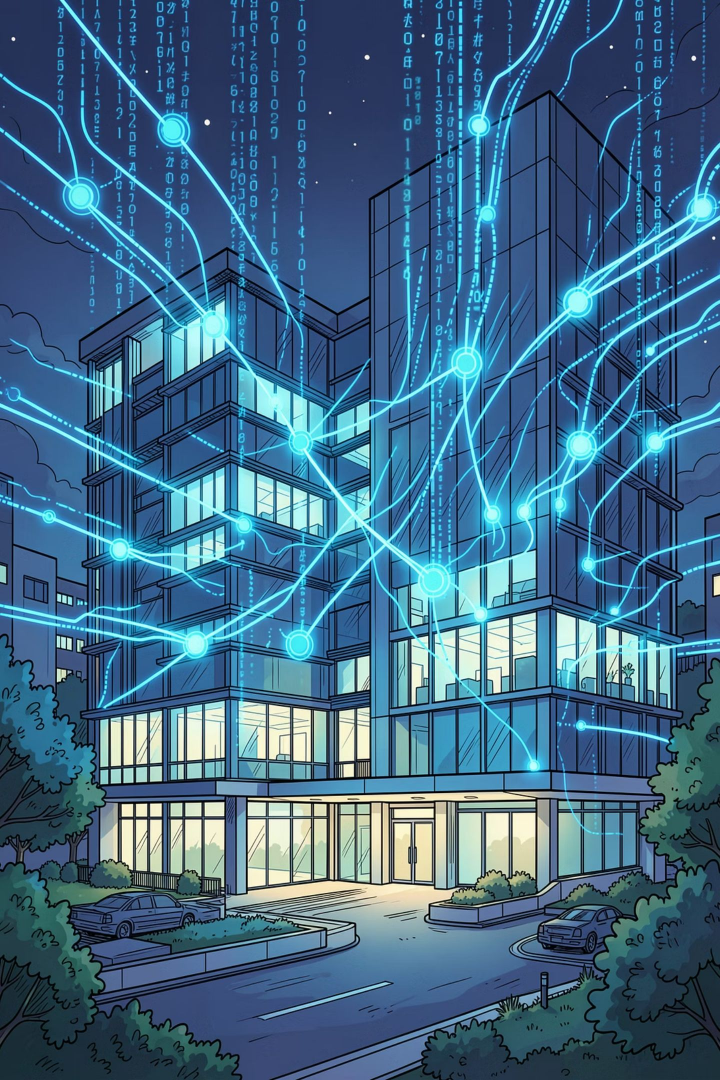


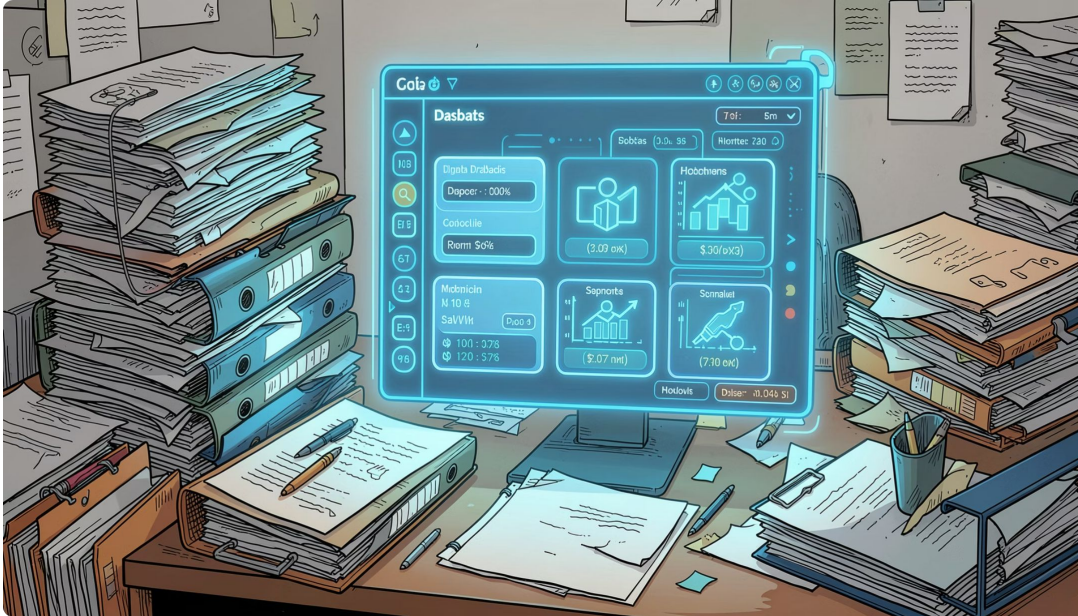


Hospital Management System: Database Architecture

Building a scalable, normalized, and secure enterprise database

*Presented by: **Dibyajyoti Chakravarti***

From Chaos to Clarity



The Problem

Manual hospital record keeping across patients, doctors, and billing creates dangerous inefficiencies duplicated data, delayed care, and critical errors.

The Objective

Design a robust relational database that ensures data integrity, automates operations, and enables efficient medical and financial reporting at enterprise scale.

Tools & Technologies



Microsoft SQL Server

Enterprise grade relational database engine powering the entire system.



SQL Server Management Studio

Primary IDE for development, debugging, and administration.



T-SQL

Transact SQL for querying, scripting, and procedural database logic.



Draw.io & GitHub

ERD design and version control for collaborative, auditable development.

Database Design & Normalization

Core Entities Mapped

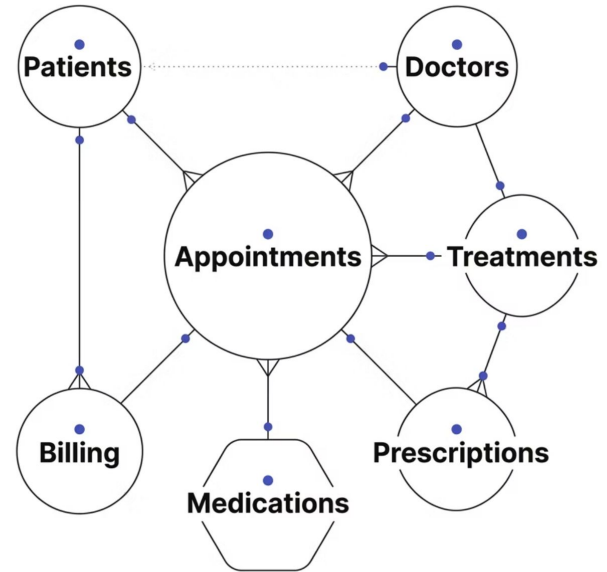
Six key entities were modeled: **Patients**, **Doctors**, **Appointments**, **Treatments**, **Prescriptions**, and **Billing**.

3NF Compliance

Strict normalization to **3rd Normal Form** eliminated redundancy and ensured referential integrity.

ERD Blueprint

A full Entity-Relationship Diagram defined all **1-to-1** and **1-to-Many** relationships.



Database Implementation: DDL & DML



Structure — DDL

Physical tables built with strict constraints to prevent bad data:

- **Primary Keys** — Unique identifiers for every record
- **Foreign Keys** — Relationship bridges between entities
- **NOT NULL, UNIQUE, CHECK** — Data quality gates

Data — DML

Populated with realistic sample data using **INSERT**, **UPDATE**, and **DELETE** operations safely and systematically.

QUERYING & REPORTING

Advanced SQL Intelligence

Complex queries extracted actionable medical and financial insights from the normalized schema.

JOINS

INNER, LEFT, RIGHT, and FULL JOINS merged data across multiple tables seamlessly.

Aggregation

GROUP BY and HAVING generated summaries for billing, patient volumes, and treatment counts.

Subqueries

Nested subqueries enabled layered logic for advanced filtering and conditional reporting.

Optimization: Views & Indexes

Views — Virtual Tables

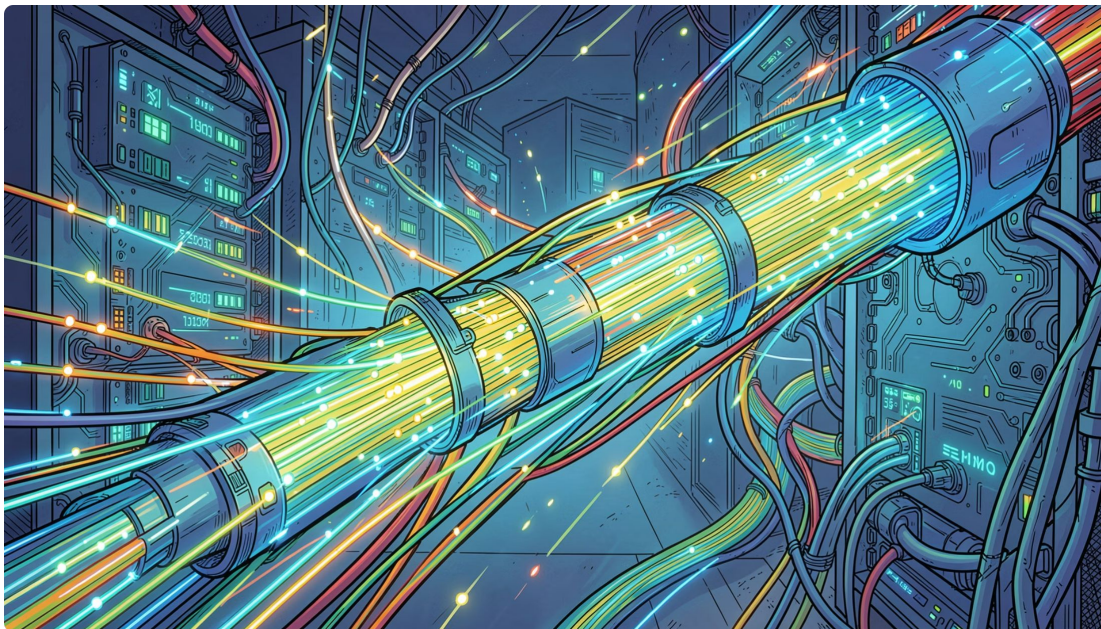
Created reusable virtual tables (e.g., "**Master Calendar**") to abstract complex multi-table joins, simplifying reporting for hospital staff.

Non-Clustered Indexes

Deployed indexes on high-search columns (e.g., **Patient Last Names**) to ensure lightning-fast retrieval during lookups and audits.



Indexes dramatically reduce query execution time on large datasets without altering the underlying data.



Automation, Safety & Security



Functions

Custom T-SQL functions automated billing calculations, eliminating manual errors.



Triggers

AFTER INSERT triggers acted as invisible audit loggers — capturing every data change automatically.



Transaction Control

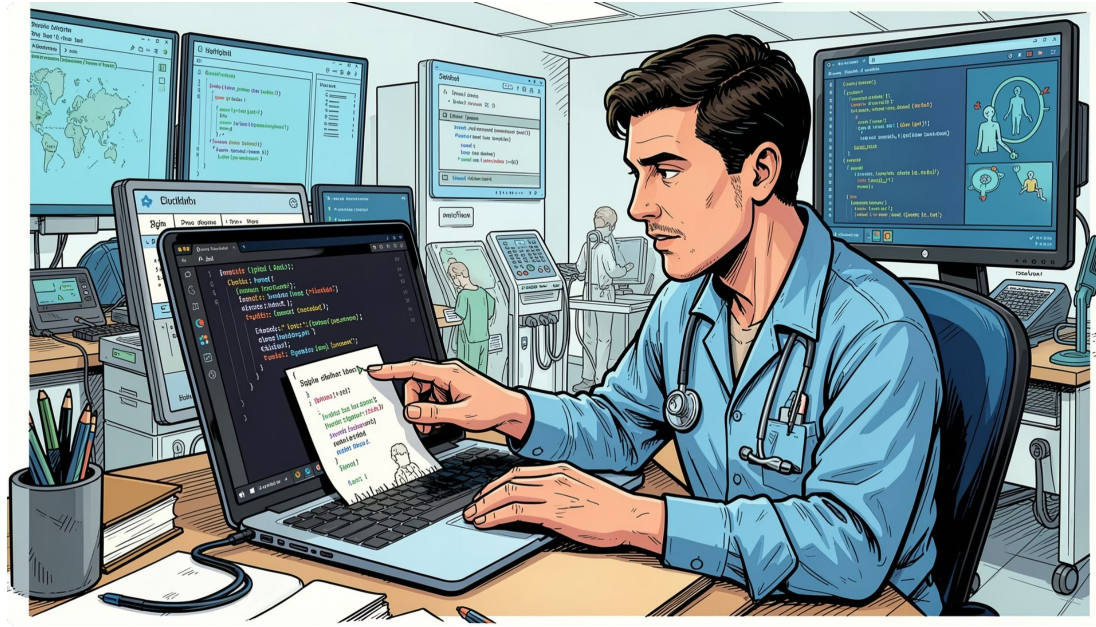
COMMIT and ROLLBACK ensured catastrophic errors could be instantly reversed, preserving data integrity.



Access Control

Role-Based Access Control (GRANT/REVOKE) restricted front-desk permissions to only what was necessary.

Deployment & Impact



What Was Delivered

All modular SQL scripts — DDL, DML, Queries, Triggers, Functions, and Indexes — packaged into a professional, version-controlled **GitHub repository**.

The Result

An enterprise-grade, highly optimized, and fully automated database capable of handling real-world hospital operations with integrity, speed, and security.



✓ Scalable · ✓ Normalized · ✓
Secure · ✓ Auditable

Thank You!

